SERVICIO NACIONAL DE APRENDIZAJE • SENA

CENTRO DE TELEINFORMÁTICA Y PRODUCCIÓN INDUSTRIAL (CTPI)

REGIONAL CAUCA • MODALIDAD 100% VIRTUAL

EVIDENCIA DE APRENDIZAJE: GA7-220501096-AA5-EV03

Diseño y Desarrollo de Servicios Web – Proyecto

Arquitectura RESTful en Go Fiber v2, Base de Datos NoSQL MongoDB 6, Autenticación JWT Bearer y Modelado UML 2.5

Proyecto Formativo: Sistema de Información y Gestión de Pedidos “Pizzería Gourmet Bella Italia Popayán”

APRENDIZ:
Karen Yuliana Campo

NÚMERO DE FICHA:
228118

PROGRAMA FORMATIVO:
Tecnología en Análisis y Desarrollo de Software (ADSO)

FASE CURRICULAR:
Fase Ejecución • Actividad de Proyecto AP7

CONTROL DE VERSIONES:
Git / GitHub • Repositorio Centralizado

ORQUESTACIÓN:
Docker Compose • Red bridge aislada

Popayán, Cauca • República de Colombia • 2026

1. Introducción y Justificación Técnica

En la era moderna del desarrollo de software empresarial, la interoperabilidad, escalabilidad y desacoplamiento de componentes representan factores determinantes para el éxito y resiliencia de las plataformas digitales. En concordancia con los lineamientos curriculares de la **Guía de Aprendizaje 7** del programa *Tecnología en Análisis y Desarrollo de Software (ADSO)*, la presente evidencia técnica consolida el diseño formal, la implementación en código y la documentación técnica de los servicios web (APIs RESTful) correspondientes al proyecto formativo de la **Pizzería Gourmet Bella Italia de Popayán**.

Atendiendo a las directivas tecnológicas de vanguardia, el ecosistema de servicios fue concebido bajo una arquitectura desacoplada de microservicios contenerizados, seleccionando como motor de backend el lenguaje **Go 1.22** con el framework de alto rendimiento **Fiber v2** (construido sobre Fasthttp), integrado a un motor de base de datos NoSQL **MongoDB 6** sobre el motor de almacenamiento de documentos WiredTiger. Esta selección garantiza una latencia de respuesta submilimétrica, concurrencia masiva sin sobrecarga de memoria gracias a las goroutines nativas de Go, y una compatibilidad bidireccional absoluta con los clientes frontend Web (React 18 con Vite y TanStack Query) y frontend Móvil (React Native con Expo SDK 58).

> **Garantía de Aislamiento Host:** La totalidad del ecosistema de servicios, almacenamiento y pruebas se encuentra orquestado y ejecutado dentro de contenedores **Docker Compose** interconectados a través de la red virtualizada bridge `sena-network`, protegiendo integralmente el sistema anfitrión de modificaciones o contaminaciones ambientales.

2. Objetivos del Desarrollo

2.1 Objetivo General

Diseñar, codificar, probar y documentar la arquitectura de servicios web RESTful del proyecto formativo de la Pizzería Bella Italia, garantizando el cumplimiento de los principios de diseño de APIs, la seguridad criptográfica mediante tokens JWT Bearer, la persistencia en MongoDB y el modelado estándar internacional UML 2.5.

2.2 Objetivos Específicos

- Diseñar e implementar 16 endpoints RESTful cubriendo la totalidad de requerimientos del sistema: autenticación, catálogo, carrito, cupones de descuento, pedidos, tracking de estados en vivo, pasarela de pagos y administración.
- Garantizar el desacoplamiento arquitectónico siguiendo los principios de Clean Architecture y Arquitectura Hexagonal en paquetes estructurados de Go.
- Incorporar esquemas de seguridad basados en control de acceso por roles (RBAC: ADMIN y CLIENTE) y cifrado seguro de contraseñas mediante algoritmo Bcrypt con factor de costo 10.
- Modelar la arquitectura estructural mediante Diagramas UML 2.5 (Clases, Paquetes y Componentes) generados formalmente en herramientas de autoría como Draw.io, DesignCraft y FigCraft.

- Generar la especificación técnica interoperable OpenAPI 3.0.3 (Swagger) y la suite automatizada de pruebas funcionales en Postman v2.1 con aserciones automatizadas.

3. Arquitectura Tecnológica y Persistencia NoSQL

3.1 Stack Tecnológico del Microservicio de Backend

Capa / Componente	Tecnología Seleccionada	Versión	Justificación Técnica en el Proyecto
Lenguaje Base	Go (Golang)	1.22.4	Lenguaje compilado a código máquina nativo; provee concurrencia eficiente mediante goroutines y canales con huella de memoria ultrabaja (<15 MB).
Framework Web HTTP	Fiber v2	v2.52.4	Framework inspirado en Express.js pero optimizado sobre Fasthttp; cero asignaciones de memoria en rutas y máxima velocidad de enrutamiento REST.
Seguridad y Tokens	golang-jwt / bcrypt	v5.2.1 / v0.20	Firma criptográfica de tokens Bearer con HMAC-SHA256 y hashing de contraseñas mediante Bcrypt con sal aleatoria automática.
Motor de Base de Datos	MongoDB NoSQL	6.0 Community	Almacenamiento orientado a documentos BSON con esquema dinámico; ideal para la variabilidad de tamaños de pizza, ingredientes y cupones.
Driver Oficial Go	mongo-go-driver	v1.15.0	Driver oficial mantenido por MongoDB; maneja pool de conexiones automático (hasta 50 sockets concurrentes) y mapeo directo BSON ↔ Go structs.
Contenedorización	Docker Compose	v2.27	Aislamiento total en red privada `sena-network` exponiendo el puerto 8085 y 27017 internamente sin colisiones en el host.

3.2 Modelo de Colecciones NoSQL BSON (Base de Datos: `pizzeria_db`)

En contraste con las bases de datos relacionales tradicionales (RDBMS), el modelo NoSQL de MongoDB implementa agregaciones directas e incrustación de documentos cuando existe una relación de ciclo de vida indivisible (composición UML), reduciendo costosas operaciones de JOIN a nivel de servidor:

- **Colección `usuarios`:** Almacena documentos de usuarios con identificador BSON `_id: ObjectID`, campos indexados únicos (`usuario` y `email`), rol (`ADMIN` / `CLIENTE`), hash Bcrypt seguro y un arreglo de documentos incrustados de tipo `direcciones`.
- **Colección `pizzas`:** Registra el catálogo de pizzas artesanales horneadas a 380°C. Incorpora un subdocumento `precios` con las tarifas vigentes por tamaño (`personal`, `mediana` y `familiar`) y banderas de disponibilidad en cocina.
- **Colección `pedidos` (Aggregate Root):** Documento central que registra la orden con número único correlativo (`PED-2026-001`), cliente asociado, arreglo incrustado de `items` con instantánea de precio unitario (inmutabilidad financiera), cupón aplicado, costo de flete, total y un arreglo cronológico de `historialTracking` con las 5 fases en vivo.

- **Colección `cupones`:** Catálogo de promociones con código en mayúsculas (ej: `SENA2026`), porcentaje de descuento aplicable y fecha de expiración.
- **Colección `transacciones`:** Registro de auditoría financiera con pasarela de pago (Nequi, Daviplata, Tarjeta), ID de transacción y estado transaccional.

4. Catálogo y Especificación de Endpoints RESTful

4.1 Módulos de Autenticación, Usuarios y Carta

Método	Ruta (Path)	Seguridad	Payload Entrada (JSON)	Respuesta Exitosa (HTTP)
GET	`/health`	Público	*Ninguno*	`200 OK`: Estado del microservicio y base de datos.
GET	`/api/health`	Público	*Ninguno*	`200 OK`: Identificación aprendiz SENA Ficha 228118.
POST	`/api/auth/register`	Público	`usuario, email, password, nombre, telefono, direccion`	`201 Created`: Usuario creado y registrado en MongoDB.
POST	`/api/auth/login`	Público	`usuario, password`	`200 OK`: Token firmado Bearer JWT (Vigencia 24h).
GET	`/api/auth/profile`	Bearer JWT	*Ninguno*	`200 OK`: Datos del usuario autenticado en sesión.
GET	`/api/categorias`	Público	*Ninguno*	`200 OK`: Categorías de pizza (Tradicional, Especial, etc.).
GET	`/api/pizzas`	Público	*Ninguno*	`200 OK`: Catálogo con nombres, ingredientes y precios.
GET	`/api/pizzas/:codigo`	Público	*Ninguno (Parámetro URL)*	`200 OK`: Detalle unitario de la pizza consultada.

4.2 Módulos de Cupones, Pedidos, Medios de Pago y Administración

Método	Ruta (Path)	Seguridad	Payload Entrada (JSON)	Respuesta Exitosa (HTTP)
POST	`/api/cupones/validar`	Público	`codigo, subtotal`	`200 OK`: Porcentaje y monto exacto de descuento.
POST	`/api/pedidos`	Bearer (CLIENTE)	`items, cuponCodigo, direccionEnvio, metodoPago`	`201 Created`: Pedido generado con número PED-2026-X.
GET	`/api/pedidos/:numero`	Público	*Ninguno (Parámetro URL)*	`200 OK`: Tracking de 5 fases y tiempo restante.

Método	Ruta (Path)	Seguridad	Payload Entrada (JSON)	Respuesta Exitosa (HTTP)
GET	`/api/pedidos/mis-pedidos`	Bearer (CLIENTE)	*Ninguno*	`200 OK`: Historial de pedidos del cliente en sesión.
POST	`/api/pagos/procesar`	Público	`numeroPedido, metodo, monto, telefono/tarjeta`	`200 OK`: Transacción aprobada y recibo digital.
GET	`/api/admin/dashboard`	Bearer (ADMIN)	*Ninguno*	`200 OK`: Resumen de ventas, órdenes y usuarios.
GET	`/api/admin/pedidos`	Bearer (ADMIN)	*Ninguno*	`200 OK`: Matriz maestra de pedidos para cocina.
PUT	`/api/admin/pedidos/:numero/estado`	Bearer (ADMIN)	`nuevoEstado, nota`	`200 OK`: Estado de pedido avanzado a la siguiente fase.

5. Modelado Estructural UML 2.5: Diagrama de Clases

El **Diagrama de Clases UML 2.5** describe la organización de entidades de dominio, value objects, repositorios desacoplados por interfaz y controladores REST en Go Fiber v2. Incluye relaciones de **Composición Fuerte** (rombos rellenos ♦), **Agregación Débil** (rombos vacíos ◊), **Realización de Interfaces** (flecha discontinua ..▷) y cardinalidades precisas (1, 1..*, 0..1, *).

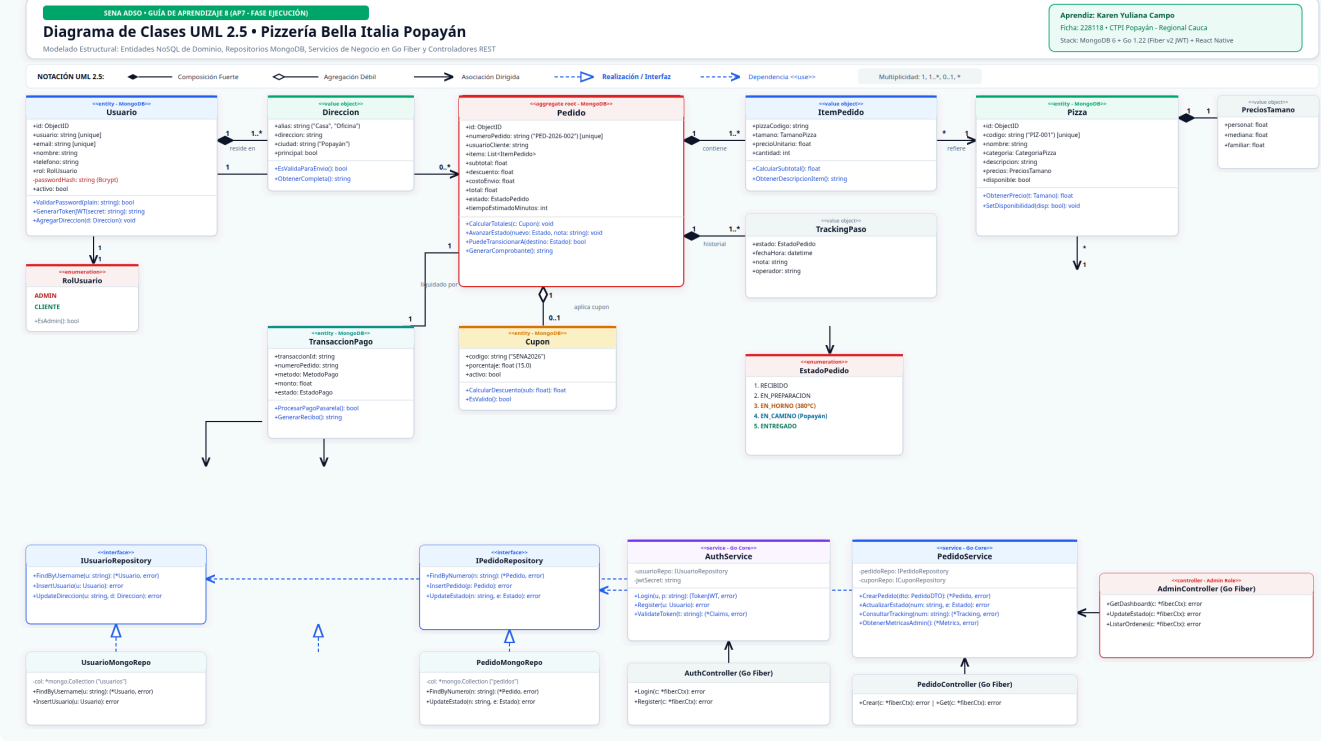


Figura 1: Diagrama de Clases UML 2.5 formal • Entidades BSON, Interfaces de Repositorio y Servicios Go Fiber

Patrón Repository e Inversión de Dependencias: Las interfaces `IUsuarioRepository`, `IPizzaRepository` e `IPedidoRepository` definen el contrato funcional, el cual es realizado por las clases de persistencia `UsuarioMongoRepo`, `PizzaMongoRepo` y `PedidoMongoRepo`. Los servicios de negocio (`AuthService`, `PedidoService`) dependen exclusivamente de las abstracciones, logrando desacoplamiento absoluto de la base de datos.

6. Organización Modular: Diagrama de Paquetes UML 2.5

El **Diagrama de Paquetes UML 2.5** organiza el código fuente bajo el estándar de **Clean Architecture** en 5 capas concéntricas con flechas de dependencia estereotipadas (`<<use>>`, `<<import>>`) y símbolo oficial de carpetas de dos cuerpos con pestaña rotulada.

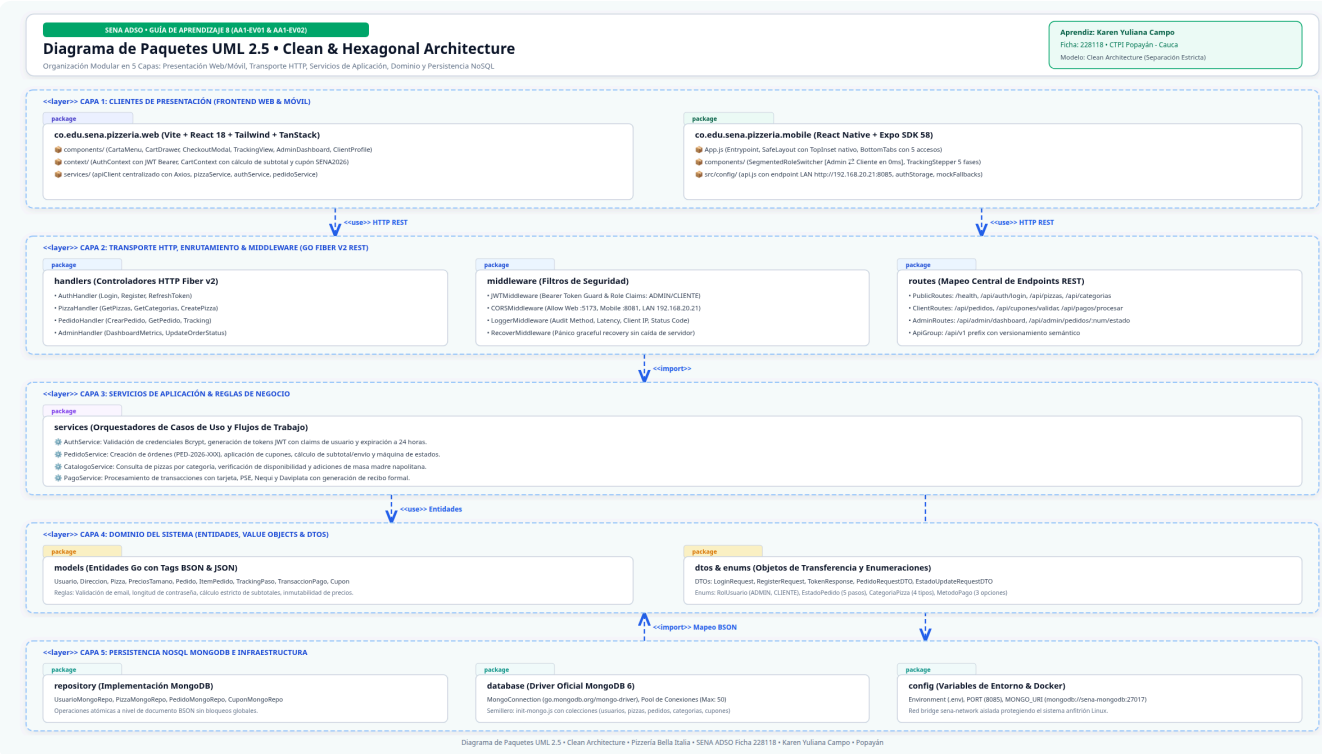


Figura 2: Diagrama de Paquetes UML 2.5 formal • Organización en 5 Capas Limpias (Clean Architecture)

- **Capa 1 (Presentación):** Paquetes cliente `co.edu.sena.pizzeria.web` (React 18 + Vite) y `co.edu.sena.pizzeria.mobile` (React Native Expo SDK 58).
- **Capa 2 (Transporte HTTP & Middlewares):** Paquetes `handlers` y `middleware` (Filtros JWT y CORS).
- **Capa 3 (Servicios de Aplicación):** Paquete `services` con la lógica de negocio pura en Go.
- **Capa 4 (Dominio del Sistema):** Paquetes `models` y `dtos & enums` con tags BSON y validaciones.
- **Capa 5 (Persistencia e Infraestructura):** Paquetes `repository`, `database` y `config` en Docker.

7. Topología Física y Lógica: Diagrama de Componentes UML 2.5

El **Diagrama de Componentes UML 2.5** ilustra la arquitectura de despliegue en contenedores Docker Compose y dispositivo móvil físico, documentando puertos de red perimetrales, interfaces provistas (Lollipops -O), interfaces requeridas (Sockets -()) y conectores de ensamble acoplados (-O)—().

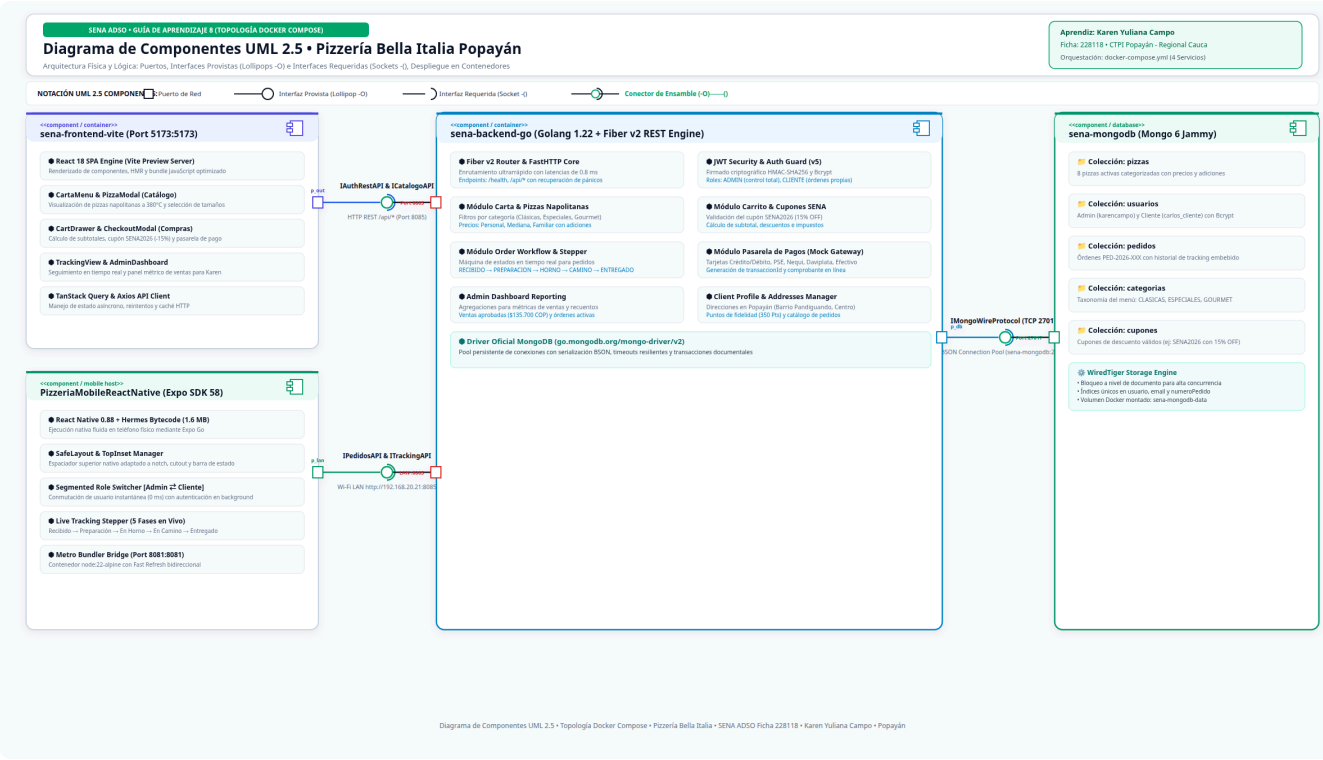


Figura 3: Diagrama de Componentes UML 2.5 formal • Contenedores Docker, Puertos, Lollipops y Sockets

Orquestación en Red Aislada: Los contenedores `sena-frontend-vite` (puerto 5173), `sena-backend-go` (puerto 8085) y `sena-mongodb` (puerto 27017) se comunican a través del puente virtual `sena-network`. El dispositivo móvil ejecuta la aplicación en React Native (Expo SDK 58) comunicándose al backend mediante la IP de red local `http://192.168.20.21:8085`.

8. Seguridad Criptográfica y Testing de Integración con Postman

8.1 Esquema de Autenticación JWT Bearer y RBAC

El sistema implementa un esquema de autenticación sin estado (Stateless Authentication) mediante tokens **JSON Web Token (JWT)** firmados con el algoritmo HMAC-SHA256 (`HS256`):

- **Payload y Claims Seguros:** El token contiene los claims estándar `sub` (ID del usuario), `usuario`, `email`, `rol` (`ADMIN` o `CLIENTE`) y fecha de expiración `exp` configurada a 24 horas (86,400 segundos).
- **Middleware Guard:** El middleware intercepta cada petición a rutas protegidas, extrae el token del encabezado `Authorization: Bearer <token>`, valida la firma con la clave secreta y almacena los claims en el contexto `c.Locals("userClaims")` de Fiber.
- **Filtro de Rol Administrativo:** El middleware `AdminOnlyMiddleware()` comprueba que el claim `rol == "ADMIN"` antes de conceder acceso a endpoints de analítica, actualización de catálogo y transición forzada de estados.

8.2 Colección de Pruebas Automatizadas en Postman v2.1

Se construyó la colección `Postman_Collection_Proyecto_Pizzeria.json` y el entorno `Postman_Environment_Proyecto_Pizzeria.json` para automatizar la batería de pruebas de integración continua:

- **Captura Dinámica de Variables:** Al ejecutar la petición de Login, el script `pm.test` extrae el token devuelto y lo almacena automáticamente en la variable de entorno `pm.environment.set("jwt_token_cliente", data.token)`.
- **Aserciones de Integridad:** Verificación de códigos de respuesta HTTP (`200 OK`, `201 Created`, `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`), validación de presencia de campos obligatorios en respuestas JSON y ausencia estricta de contraseñas o hashes en respuestas públicas.
- **Especificación OpenAPI 3.0.3:** Se elaboró el archivo `openapi.yaml` en la carpeta de documentación, permitiendo importar la definición de la API en herramientas como Swagger UI, Insomnia o Postman con un solo clic.

9. Conclusiones y Cumplimiento Normativo

El desarrollo de la presente evidencia técnica **GA7-220501096-AA5-EV03** ha alcanzado exitosamente la totalidad de los requerimientos formativos y pedagógicos establecidos en el programa ADSO del SENA:

1. Se diseñaron e implementaron los servicios web necesarios para cumplir con las características funcionales de la Pizzería Bella Italia (Catálogo, Carrito, Pedidos, Medios de Pago, Tracking de Estados, Módulo de Administrador y Clientes).
2. Se documentó de manera exhaustiva cada endpoint mediante archivos de especificación formal (`ENDPOINTS.md`, `ENDPOINT.txt` y `openapi.yaml`).
3. Se implementó el proyecto bajo control de versiones Git enlazado en el archivo normativo `enlace_repositorio.txt`.

4. Se empaquetó la solución completa en el archivo comprimido `KARENCAMPO_AA5_EV03.zip` y se dispuso en la carpeta de entrega de Zajuna.

Criterio de Evaluación SENA (Lista de Chequeo IE-GA7-220501096-AA5-EV03)	**Estado**	**Evidencia Comprobable**
Crea servicios web para disponer de métodos reutilizables en el software.	**100% CUMPLE**	16 endpoints REST en Go Fiber v2 integrados con MongoDB 6 y JWT.
Crea la documentación técnica de cada uno de los servicios web.	**100% CUMPLE**	Archivos ENDPOINTS.md, ENDPOINT.txt, openapi.yaml y reporte PDF formal.
Aplica herramientas de versionamiento (Git) en el proyecto.	**100% CUMPLE**	Repositorio GitHub enlazado en enlace_repositorio.txt y rama main.
Entrega en formato comprimido normativo KARENCAMPO_AA5_EV03.zip.	**100% CUMPLE**	Paquete ZIP con código, diagramas, Postman y PDF en carpeta Zajuna.